



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 04

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 19/09/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para este ejercicio utilicé como base el código que creamos en clase, donde creamos la base del brazo, el primer tubo, una segunda articulación y otro tubo al cual le agregué una tercera articulación para la pinza que hice con dos pirámides triangulares.

```
//Articulacion 3 Gris
model = modelaux;
model = glm::translate(model, glm::vec3(2.3f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f)); // H
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
//Garra Naranja
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.3f, 0.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 1.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.5f, 0.0f); //Naranja
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.01f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 1.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.5f, 0.0f); //Naranja
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

Para realizar la llanta delantera en +Z, usé dos cubos azules para hacer una “L” donde se incrusta el tubo (cubo alargado) amarillo, que en su borde se incluye la articulación de la llanta que las hace girar en Y, esta articulación tiene una base cúbica alargada que conecta con la pata color morado (son dos cubos unidos y rotados en diferentes ángulos), que se unen con el eje de la llanta que la hará rodar. La llanta (un cilindro) gira en el eje Z hacia enfrente, es un giro negativo porque si es positivo giraría hacia atrás.

```

//Llanta Frontal +Z
//Base Azul
model = modelaux2;
model = glm::translate(model, glm::vec3(-4.0f, 0.25f, 5.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.25f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
//Tubo Amarillo
model = glm::translate(model, glm::vec3(-2.0f, -0.9f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 4.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 0.0f); //Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();

```

```

//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, -1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 1.0f, 0.0f)); // J
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.5f, -0.5f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.25f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();

```

```

//Pata Morada
model = modelaux;
model = glm::translate(model, glm::vec3(-0.5f, -0.4f, -0.85f));
modelaux = model;
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.6f, 2.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.0f, 0.5f); //Morado
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
model = modelaux;
model = glm::translate(model, glm::vec3(-0.3f, -1.0f, -0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-150.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.6f, 2.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.0f, 0.5f); //Morado
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();

```

```
//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(0.2f, -0.3f, 1.1f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 2.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

```
//Rueda 1 Verde
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.5f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Para la llanta en -Z usé las mismas figuras e instancias, ajustadas para hacer un efecto espejo con las de +Z, ambas llantas se mueven al mismo tiempo y rotan al mismo tiempo sus ejes de la siguiente manera.

```
//Llanta Frontal -Z
//Base Azul
model = modelaux2;
model = glm::translate(model, glm::vec3(-4.0f, 0.25f, -5.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
model = glm::translate(model, glm::vec3(0.0f, 1.0f, -0.25));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.5));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
//Tubo Amarillo
model = glm::translate(model, glm::vec3(-2.0f, -0.9f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 4.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 0.0f); //Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
```

```

//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, -1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 1.0f, 0.0f)); // J
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.5f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.25f, 1.5));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();

```

```

//Pata Morada
model = modelaux;
model = glm::translate(model, glm::vec3(-0.5f, -0.4f, 0.85f));
modelaux = model;
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.6f, 2.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.0f, 0.5f); //Morado
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
model = modelaux;
model = glm::translate(model, glm::vec3(-0.3f, -1.0f, -0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-150.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.6f, 2.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.0f, 0.5f); //Morado
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();

```

```

//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(0.2f, -0.3f, -1.1f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, -1.0f, 0.0f)); // K
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 2.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();

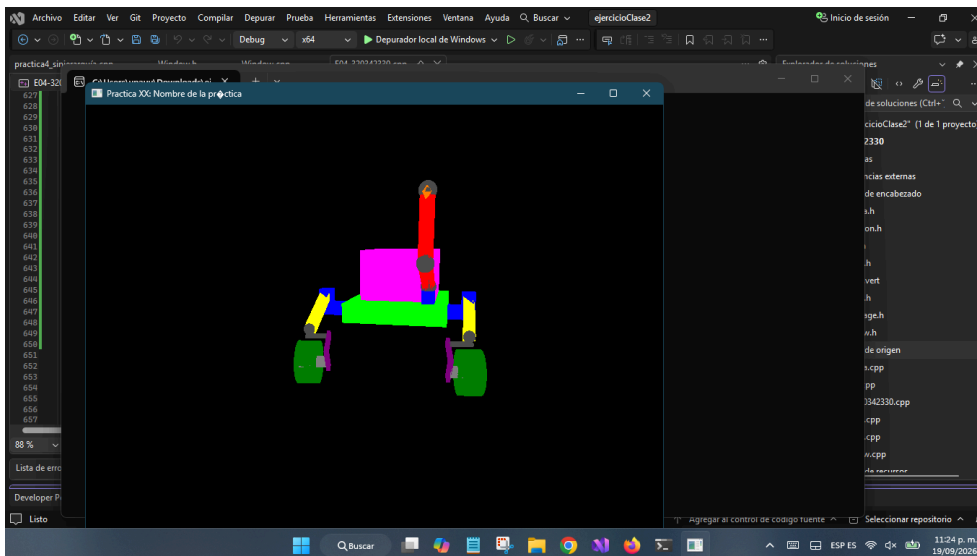
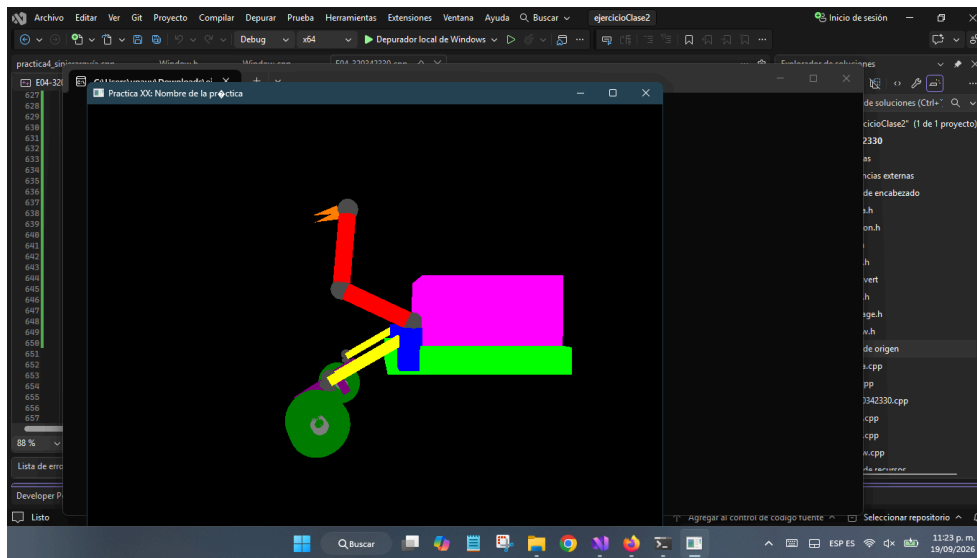
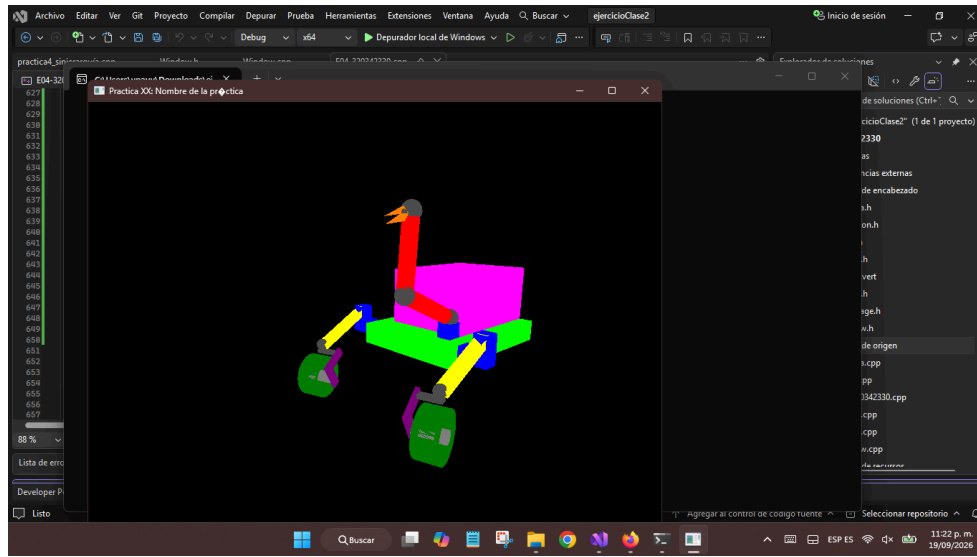
```

```

//Rueda 1 Verde
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.5f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();

```

La ejecución final se muestra a continuación:



2. Problemas presentados.

No se presentó ningún problema durante la realización de la práctica más allá del análisis del modelo original para poderlo recrear.

3. Conclusión:

Gracias al desarrollo de esta práctica comprendí las bases del modelado jerárquico y fui capaz de crear figuras que pueden rotar alrededor de otras, me pareció muy útil para crear las primeras partes del Rover y me agradó poder usar las figuras disponibles para crear una versión medianamente fiel al modelo proporcionado por el profesor. La explicación fue muy buena y entendí todos los conceptos en la clase.